

BÁO CÁO KẾT QUẢ THỰC TẬP

Đề tài: Xây dựng và Hoàn thiện Hệ thống IoT Quản lý Khu vườn Thông minh (Smart Garden IoT Management System)

I. LÝ THUYẾT CỐT LÕI & KIẾN TRÚC HỆ THỐNG

1. Kiến trúc Đa Giao thức (NestJS Hybrid Architecture)

Hệ thống được thiết kế theo mô hình Hybrid Architecture – kết hợp đồng thời nhiều giao thức truyền thông trong cùng một ứng dụng, mỗi giao thức đảm nhiệm một lớp trách nhiệm riêng:

1.1. RESTful API (HTTP Layer):

Đảm nhiệm các tác vụ quản lý tài nguyên theo chuẩn CRUD đối với người dùng, khu vườn, nông sản, bảng giá và doanh thu. Toàn bộ luồng request đều được kiểm tra thông qua ValidationPipe (với whitelist: true, forbidNonWhitelisted: true) nhằm đảm bảo dữ liệu đầu vào hợp lệ và chặn các trường trái phép, đồng thời sử dụng PrismaClientExceptionFilter toàn cục để chuẩn hóa cơ chế trả lỗi HTTP, giúp hệ thống xử lý lỗi nhất quán và dễ dàng bảo trì.

1.2. MQTT Microservice (IoT Telemetry Layer):

Áp dụng mô hình Publish/Subscribe – một trong những mô hình truyền thông tối ưu nhất cho các hệ thống IoT hoạt động trên mạng băng thông thấp. Lớp này chịu trách nhiệm:

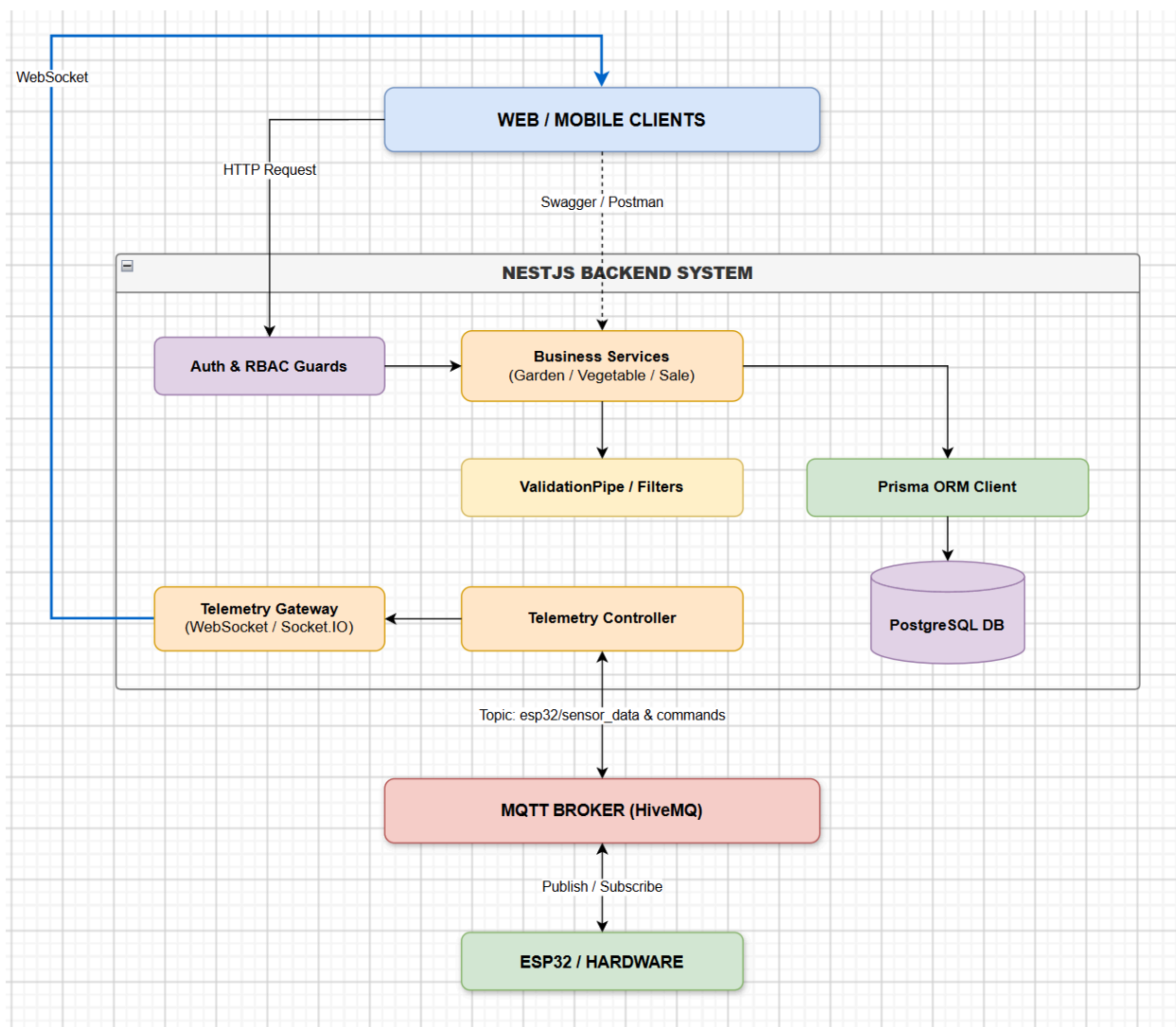
- tiếp nhận dữ liệu cảm biến từ vi điều khiển ESP32 thông qua topic esp32/sensor_data,
- gửi lệnh điều khiển từ hệ thống xuống thiết bị phần cứng qua topic esp32/commands.

1.3. WebSocket Gateway (Real-time Streaming Layer):

Duy trì kết nối Full-Duplex thông qua Socket.IO, cho phép máy chủ chủ động đẩy dữ liệu tới client ngay khi có sự thay đổi mà không cần cơ chế polling liên tục. Nhờ đó:

- thông số cảm biến môi trường được cập nhật tức thì lên giao diện Dashboard,
- sự kiện cảnh báo ngưỡng (critical_alert) được phát tới client ngay lập tức,
- giảm tải đường truyền và tăng khả năng phản hồi theo thời gian thực.

ẢNH 1: Kiến trúc tổng thể hệ thống (REST / MQTT / WebSocket)



2. Quản trị Dữ liệu với PostgreSQL, Prisma ORM & Transaction

2.1. Mô hình hóa quan hệ dữ liệu (Data Modeling)

Hệ thống sử dụng PostgreSQL làm hệ quản trị cơ sở dữ liệu chính và Prisma ORM để thao tác dữ liệu một cách an toàn, trực quan. Mô hình dữ liệu được thiết kế với các mối quan hệ chính như sau:

- **User ↔ Garden (1 – n):**

Một người dùng có thể sở hữu nhiều khu vườn; quan hệ này giúp phân định rõ quyền sở hữu tài nguyên và làm cơ sở cho việc kiểm soát truy cập.

- **Garden ↔ Vegetable (1 – n) ↔ VegetablePrice (1 – n):**

Mỗi khu vườn có nhiều loại nông sản; mỗi nông sản có thể trải qua nhiều lần thay đổi giá, được lưu trữ trong bảng VegetablePrice để quản lý vòng đời nông sản và lịch sử biến động giá theo thời gian.

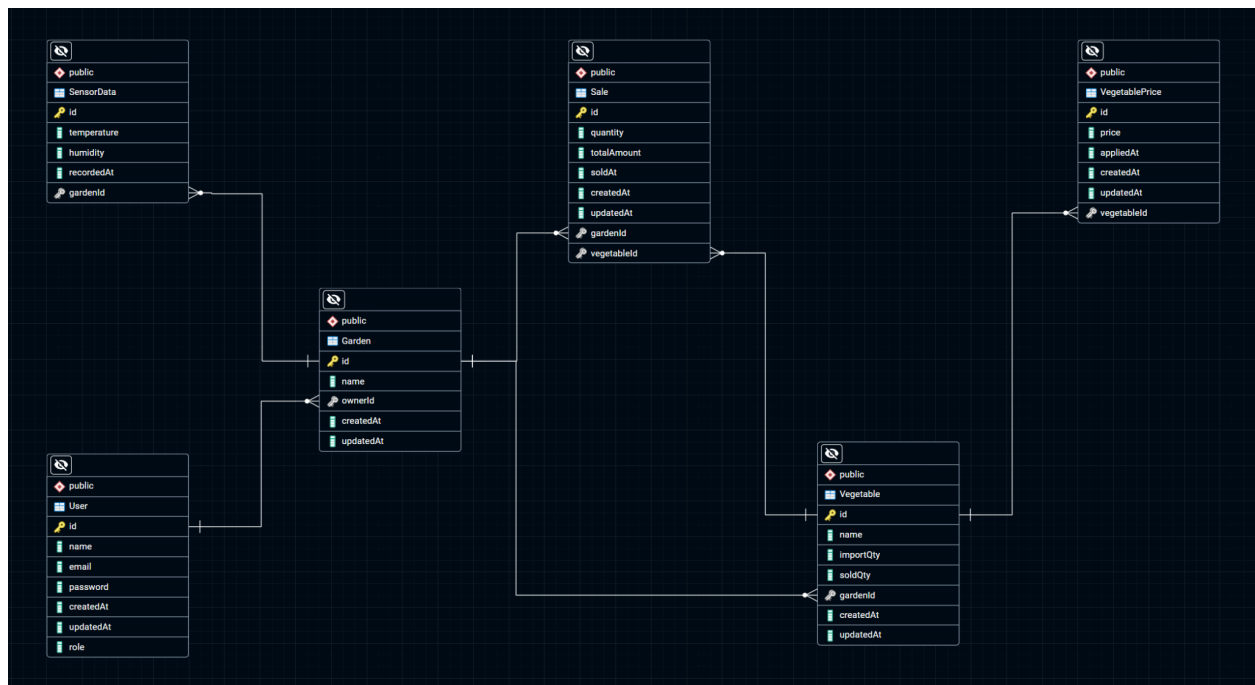
- **Garden/Vegetable ↔ Sale (1 – n):**

Ghi nhận toàn bộ giao dịch bán hàng, phục vụ truy xuất doanh thu và phân tích hiệu quả kinh doanh.

- **Garden ↔ SensorData (1 – n):**

Lưu trữ chuỗi dữ liệu thời gian (time-series data) về nhiệt độ, độ ẩm thu thập từ phần cứng IoT, phục vụ công tác giám sát môi trường khu vườn.

ẢNH 2: Mô hình cơ sở dữ liệu (ERD)



2.2. Quy ước định danh dữ liệu nghiệp vụ

Trong hệ thống, mỗi bản ghi Vegetable đại diện cho một mặt hàng cụ thể tại một khu vườn xác định, được định danh thông qua **cặp khóa** ``vegetableId` + `gardenId`` trên URL. Điều này có ý nghĩa quan trọng trong nghiệp vụ:

- cùng một loại rau xuất hiện ở hai khu vườn khác nhau vẫn được quản lý như hai bản ghi độc lập,
- mỗi bản ghi có giá bán, số lượng nhập và tình trạng tồn kho riêng biệt,

- khi thêm mới hoặc xử lý dữ liệu, hệ thống luôn yêu cầu truyền đầy đủ gardenId qua URL hoặc Body để xác định chính xác bản ghi thuộc khu vườn nào, tránh nhầm lẫn giữa các khu vườn.

2.3. Prisma Interactive Transaction

Hệ thống sử dụng `prisma.transaction()` để đảm bảo tính nhất quán tuyệt đối của dữ liệu theo nguyên tắc ACID. Trong quá trình xử lý đơn hàng, hệ thống thực hiện đồng thời trong một phiên giao dịch nguyên tử các bước:

1. kiểm tra sự tồn tại của nông sản và đối chiếu gardenId của nông sản với khu vườn yêu cầu,
2. kiểm tra số lượng tồn kho ($\text{importQty} - \text{soldQty}$),
3. đối chiếu đơn giá mới nhất,
4. trừ số lượng trong kho,
5. ghi nhận hóa đơn bán hàng.

Nếu bất kỳ bước nào xảy ra lỗi, toàn bộ giao dịch sẽ bị rollback, đảm bảo dữ liệu không bao giờ rơi vào trạng thái sai lệch (ví dụ: trừ kho nhưng không tạo được hóa đơn).

ẢNH 3: Đoạn code prisma.transaction() trong src/sale/sale.service.ts

```
15 return this.prisma.$transaction(async (tx) => {
16   // 1. Fetch vegetable details with its latest set price
17   const veg = await tx.vegetable.findUnique({
18     where: { id: dto.vegetableId },
19     include: { prices: { orderBy: { appliedAt: 'desc' }, take: 1 } },
20   });
21
22   if (!veg) throw new NotFoundException(`Vegetable with ID ${dto.vegetableId} not found.`);
23   if (veg.gardenId !== dto.gardenId) {
24     throw new BadRequestException('Specified vegetable does not belong to this garden.');
```

```
25   }
26
27   // 2. Stock check
28   const availableStock = veg.importQty - veg.soldQty;
29   if (availableStock < dto.quantity) {
30     throw new BadRequestException(`Insufficient stock. Available: ${availableStock}, requested: ${dto.quantity}.`);
31   }
32
33   // 3. Price check
34   if (veg.prices.length === 0) {
35     throw new BadRequestException('Vegetable does not have a price set.');
```

```
36   }
37
38   const currentPrice = veg.prices[0].price;
39   const totalAmount = currentPrice * dto.quantity;
40
41   // 4. Update sold quantity in Vegetable table
42   await tx.vegetable.update({
43     where: { id: dto.vegetableId },
44     data: { soldQty: veg.soldQty + dto.quantity },
45   });
46
47   // 5. Create Sale transaction record
48   return tx.sale.create({
49     data: {
50       quantity: dto.quantity,
51       totalAmount,
52       gardenId: dto.gardenId,
53       vegetableId: dto.vegetableId,
54     },
55     include: {
56       garden: { select: { id: true, name: true } },
57       vegetable: { select: { id: true, name: true } },
58     },
59   });
60 });
```

3. Bảo mật Hệ thống & Phân quyền Vai trò (RBAC)

3.1. Xác thực JWT & Guard Strategy:

Hệ thống tích hợp AuthGuard và JwtStrategy để xác thực người dùng thông qua JSON Web Token (JWT). Sau khi xác thực, thông tin định danh của người dùng được trích xuất và gắn vào req.user thông qua decorator @CurrentUser() để phục vụ các tầng kiểm tra quyền tiếp theo.

3.2. Kiểm soát Truy cập RBAC (RolesGuard):

Hệ thống phân chia hai cấp độ người dùng:

- Role.ADMIN: nắm quyền quản trị toàn hệ thống, bao gồm quản lý tài khoản và truy cập dữ liệu tổng hợp,

- Role.USER: chỉ có quyền thao tác trên khu vườn và nông sản do mình làm chủ.

Trong GardenService, sau khi tìm thấy khu vườn, hệ thống kiểm tra quyền sở hữu: nếu không phải ADMIN và garden.ownerId khác user.id, request sẽ bị từ chối với lỗi 403 Forbidden.

Phân quyền hai tầng (Two-tier Permission Model):

- Tầng quản trị hệ thống (ADMIN): được phép xem toàn bộ dữ liệu (khu vườn, nông sản, giá, doanh thu, người dùng), cập nhật/xóa khu vườn của mọi người dùng nhằm phục vụ công tác giám sát và dọn dẹp dữ liệu vi phạm, điều khiển thiết bị IoT, quản lý tài khoản. Các quyền này chỉ được thực hiện vì các mục đích quản trị hợp lệ, không nhằm can thiệp vào hoạt động kinh doanh thường nhật của người dùng.
- Tầng nghiệp vụ sở hữu (Owner-only): các thao tác thêm/sửa/xóa nông sản, thiết lập giá và tạo giao dịch bán hàng chỉ được thực hiện bởi chính chủ sở hữu khu vườn. Ngay cả ADMIN cũng bị hệ thống từ chối với lỗi 403 Forbidden khi cố can thiệp vào dữ liệu kinh doanh của người dùng khác, bởi các thao tác này ảnh hưởng trực tiếp đến tồn kho, giá bán và doanh thu của chủ khu vườn.

Việc tách bạch hai tầng quyền đảm bảo dữ liệu nghiệp vụ luôn thuộc quyền kiểm soát của chủ khu vườn, đồng thời vẫn duy trì khả năng quản trị, giám sát và xử lý vi phạm cấp hệ thống cho ADMIN.

3.3. Phòng chống Leo thang Đặc quyền (Privilege Escalation):

Để ngăn chặn người dùng tự ý nâng quyền hoặc sửa các trường nhạy cảm, hệ thống:

- bóc tách DTO nghiêm ngặt theo từng nghiệp vụ (ví dụ UpdateUserDto chỉ cho phép name và password),
- áp dụng whitelist: true + forbidNonWhitelisted: true để loại bỏ hoàn toàn các trường dữ liệu không nằm trong DTO (như tự ý gửi role khi cập nhật hồ sơ),
- kiểm soát quyền sở hữu ngay trong tầng Service trước khi thực hiện thao tác cập nhật/xóa.

II. NỘI DUNG THỰC HIỆN CHI TIẾT

Tuần 7: Tái cấu trúc Cơ sở dữ liệu & Xây dựng Khung Ứng dụng Cơ bản

Mục tiêu: Thiết lập lại toàn bộ nền tảng dữ liệu và xây dựng các module cốt lõi của ứng dụng.

Công việc đã thực hiện:

- Khởi tạo cơ sở dữ liệu độc lập smartgarden_db trên PostgreSQL, tách biệt hoàn toàn với database của các tuần trước để tránh xung đột dữ liệu.
- Cấu hình Prisma qua prisma.config.ts và thiết lập schema.prisma chuẩn hóa 6 bảng nghiệp vụ chính: User, Garden, Vegetable, VegetablePrice, Sale, SensorData.
- Hoàn thiện AuthModule với các chức năng: đăng ký tài khoản, đăng nhập, băm mật khẩu bằng Bcrypt, cấp phát JWT.
- Phát triển UsersModule phục vụ công tác quản trị tài khoản an toàn (chỉ cho phép cập nhật name/password, phân quyền ADMIN khi liệt kê người dùng).
- Xây dựng GardenModule và VegetableModule:
 - cung cấp trọn bộ CRUD cho khu vườn và nông sản theo cấu trúc **lồng (nested)**:
/gardens/:gardenId/vegetables,
 - kiểm soát tồn kho với ràng buộc nghiệp vụ soldQty <= importQty,
 - xây dựng cụm sub-endpoints quản lý giá rau:
 - POST /gardens/:gardenId/vegetables/:vegetableId/price
 - PUT /gardens/:gardenId/vegetables/:vegetableId/price
 - GET /gardens/:gardenId/vegetables/:vegetableId/price
 - DELETE /gardens/:gardenId/vegetables/:vegetableId/price
- Cấu hình ConfigModule global, setGlobalPrefix('api'), kiểm tra biến môi trường bắt buộc (DATABASE_URL, JWT_SECRET) ngay khi khởi động.

ẢNH 4: Log khởi động server thành công (Nest application successfully started)

```
PS D:\GitHub\YootexIoT_Intern\week7\my-app> npm start run:dev
npm notice run my-app@0.0.1 start
npm notice run nest start run:dev
[Nest] 33900 - 09/07/2026, 10:38:58 AM LOG [NestFactory] Starting Nest application...
[Nest] 33900 - 09/07/2026, 10:38:59 AM LOG [InstanceLoader] PassportModule dependencies initialized +24ms
[Nest] 33900 - 09/07/2026, 10:38:59 AM LOG [InstanceLoader] PrismaModule dependencies initialized +1ms
[Nest] 33900 - 09/07/2026, 10:38:59 AM LOG [InstanceLoader] ConfigHostModule dependencies initialized +1ms
[Nest] 33900 - 09/07/2026, 10:38:59 AM LOG [InstanceLoader] DiscoveryModule dependencies initialized +0ms
[Nest] 33900 - 09/07/2026, 10:38:59 AM LOG [InstanceLoader] AppModule dependencies initialized +255ms
[Nest] 33900 - 09/07/2026, 10:38:59 AM LOG [InstanceLoader] ConfigModule dependencies initialized +0ms
[Nest] 33900 - 09/07/2026, 10:38:59 AM LOG [InstanceLoader] ConfigModule dependencies initialized +1ms
[Nest] 33900 - 09/07/2026, 10:38:59 AM LOG [InstanceLoader] ScheduleModule dependencies initialized +0ms
[Nest] 33900 - 09/07/2026, 10:38:59 AM LOG [InstanceLoader] ClientsModule dependencies initialized +1ms
[Nest] 33900 - 09/07/2026, 10:38:59 AM LOG [InstanceLoader] JwtModule dependencies initialized +0ms
[Nest] 33900 - 09/07/2026, 10:38:59 AM LOG [InstanceLoader] UsersModule dependencies initialized +1ms
[Nest] 33900 - 09/07/2026, 10:38:59 AM LOG [InstanceLoader] GardenModule dependencies initialized +0ms
[Nest] 33900 - 09/07/2026, 10:38:59 AM LOG [InstanceLoader] VegetableModule dependencies initialized +1ms
[Nest] 33900 - 09/07/2026, 10:38:59 AM LOG [InstanceLoader] SaleModule dependencies initialized +0ms
[Nest] 33900 - 09/07/2026, 10:38:59 AM LOG [InstanceLoader] PriceModule dependencies initialized +0ms
[Nest] 33900 - 09/07/2026, 10:38:59 AM LOG [InstanceLoader] AuthModule dependencies initialized +0ms
[Nest] 33900 - 09/07/2026, 10:38:59 AM LOG [InstanceLoader] TelemetryModule dependencies initialized +0ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [NestMicroservice] Nest microservice successfully started +1564ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RoutesResolver] AppController {/api}: +34ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RouterExplorer] Mapped {/api/hello, GET} route +3ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RoutesResolver] AuthController {/api/auth}: +0ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RouterExplorer] Mapped {/api/auth/register, POST} route +1ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RouterExplorer] Mapped {/api/auth/login, POST} route +0ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RoutesResolver] UserController {/api/users}: +1ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RouterExplorer] Mapped {/api/users, GET} route +1ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RouterExplorer] Mapped {/api/users, PATCH} route +0ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RouterExplorer] Mapped {/api/users/:id, GET} route +1ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RouterExplorer] Mapped {/api/users/:id, DELETE} route +1ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RoutesResolver] TelemetryMqttController {/api}: +0ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RoutesResolver] TelemetryHttpController {/api/device}: +1ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RouterExplorer] Mapped {/api/device/command, POST} route +0ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RoutesResolver] GardenController {/api/gardens}: +0ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RouterExplorer] Mapped {/api/gardens, POST} route +1ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RouterExplorer] Mapped {/api/gardens, GET} route +1ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RouterExplorer] Mapped {/api/gardens/:id, GET} route +0ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RouterExplorer] Mapped {/api/gardens/:id, PUT} route +1ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RouterExplorer] Mapped {/api/gardens/:id, DELETE} route +1ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RoutesResolver] VegetableController {/api/gardens/:gardenId/vegetables}: +0ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RouterExplorer] Mapped {/api/gardens/:gardenId/vegetables, POST} route +1ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RouterExplorer] Mapped {/api/gardens/:gardenId/vegetables, GET} route +1ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RouterExplorer] Mapped {/api/gardens/:gardenId/vegetables/:vegetableId, PUT} route +1ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RouterExplorer] Mapped {/api/gardens/:gardenId/vegetables/:vegetableId/price, POST} route +0ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RouterExplorer] Mapped {/api/gardens/:gardenId/vegetables/:vegetableId/price, PUT} route +1ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RouterExplorer] Mapped {/api/gardens/:gardenId/vegetables/:vegetableId/price, GET} route +1ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RouterExplorer] Mapped {/api/gardens/:gardenId/vegetables/:vegetableId/price, DELETE} route +1ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RoutesResolver] SaleController {/api/sales}: +0ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RouterExplorer] Mapped {/api/sales, POST} route +2ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RouterExplorer] Mapped {/api/sales, GET} route +1ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RouterExplorer] Mapped {/api/sales/:id, GET} route +0ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RoutesResolver] PriceController {/api}: +0ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RouterExplorer] Mapped {/api/price, GET} route +1ms
[Nest] 33900 - 09/07/2026, 10:39:00 AM LOG [RouterExplorer] Mapped {/api/all/price, GET} route +1ms
[Nest] 33900 - 09/07/2026, 10:39:01 AM LOG [PrismaService] Connected to PostgreSQL database successfully.
[Nest] 33900 - 09/07/2026, 10:39:01 AM LOG [NestApplication] Nest application successfully started +4ms
[Nest] 33900 - 09/07/2026, 10:39:01 AM LOG [Bootstrap] HTTP server running on: http://localhost:3000/api
[Nest] 33900 - 09/07/2026, 10:39:01 AM LOG [Bootstrap] Swagger UI available at: http://localhost:3000/api/docs
```

Kết quả: Hệ thống có nền tảng vững chắc để triển khai các nghiệp vụ phức tạp hơn ở các tuần tiếp theo.

Tuần 8: Quản lý Giao dịch Bán hàng & Phân tích Doanh thu

Mục tiêu: Xây dựng nghiệp vụ bán hàng an toàn và hệ thống phân tích tài chính.

Công việc đã thực hiện:

- Xây dựng SaleModule với nghiệp vụ bán nông sản sử dụng `prisma.transaction()`:
 - tự động kiểm tra nông sản thuộc đúng khu vườn (`vegetable.gardenId === dto.gardenId`),
 - kiểm tra tồn kho `availableStock = importQty - soldQty`,
 - áp đơn giá mới nhất, tính `totalAmount = price × quantity`,
 - trừ số lượng và ghi nhận hóa đơn một cách đồng bộ.
- Thiết lập PriceModule độc lập nhằm giải quyết bài toán phân tích tài chính:
 - GET /price: truy xuất lịch sử thay đổi đơn giá, hỗ trợ lọc theo chu kỳ day, week, month,
 - GET /all/price: tổng hợp doanh thu toàn hệ thống và phân rã chi tiết doanh số theo từng mặt hàng bằng `prisma.sale.aggregate()` và `groupBy()`.

Kết quả: Hệ thống ghi nhận chính xác các giao dịch bán hàng, đồng thời cung cấp góc nhìn tổng quan về doanh thu phục vụ công tác quản lý.

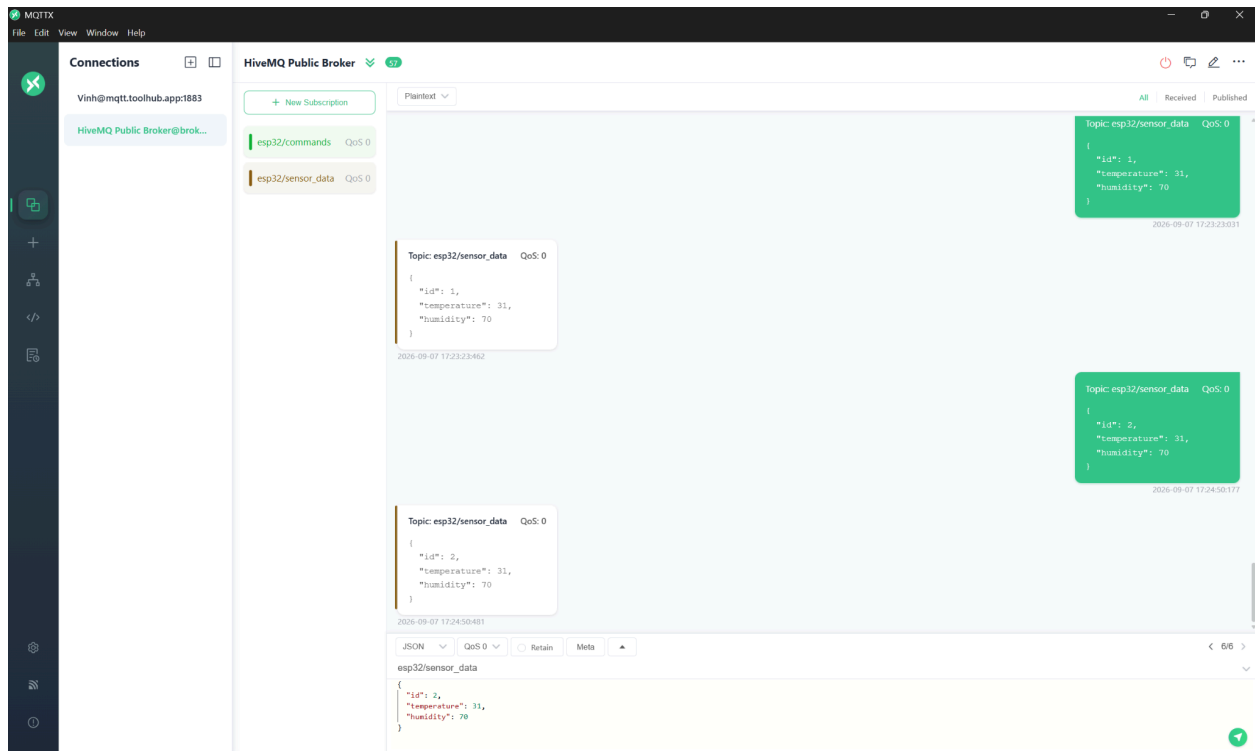
Tuần 9: Tích hợp Giao thức IoT (MQTT Telemetry & Điều khiển Thiết bị)

Mục tiêu: Kết nối hệ thống backend với phần cứng IoT, hoàn thiện luồng dữ liệu hai chiều.

Công việc đã thực hiện:

- Kết nối ứng dụng tới MQTT Broker (HiveMQ) thông qua `@nestjs/microservices`; địa chỉ broker được cấu hình linh hoạt qua biến môi trường `MQTT_URL` (mặc định `mqt://broker.hivemq.com:1883`).
- Xây dựng TelemetryMqttController:
 - lắng nghe topic `esp32/sensor_data`,
 - kiểm tra sự tồn tại của khu vườn trước khi ghi dữ liệu,
 - ghi dữ liệu chuỗi thời gian vào bảng `SensorData`,
 - phát sóng dữ liệu tới client qua TelemetryGateway sử dụng `Socket.IO`.
- Xây dựng TelemetryHttpController:
 - cung cấp endpoint POST `/device/command`,
 - nhận dữ liệu `{ "gardenId", "userId", "led1State" }`,
 - publish trực tiếp xuống topic `esp32/commands` để bật/tắt hệ thống đèn.

ẢNH 5: Log MQTT nhận dữ liệu cảm biến từ ESP32 (topic esp32/sensor_data)



```
[Nest] 23956 - 09/07/2026, 5:22:25 PM LOG [TelemetryMqttController] Received MQTT packet for Garden #11: 30°C, 60%
[Nest] 23956 - 09/07/2026, 5:22:25 PM WARN [TelemetryMqttController] Ignored packet: Garden #11 does not exist in DB.
[Nest] 23956 - 09/07/2026, 5:22:56 PM LOG [TelemetryMqttController] Received MQTT packet for Garden #1: 30°C, 60%
[Nest] 23956 - 09/07/2026, 5:23:23 PM LOG [TelemetryMqttController] Received MQTT packet for Garden #1: 31°C, 70%
[Nest] 23956 - 09/07/2026, 5:24:50 PM LOG [TelemetryMqttController] Received MQTT packet for Garden #2: 31°C, 70%
```



```
[SENSOR] Published -> Temp: 32.1 °C | Hum: 74.0% | Packet #131
[SENSOR] Published -> Temp: 30.8 °C | Hum: 73.1% | Packet #132
[SENSOR] Published -> Temp: 31.7 °C | Hum: 72.3% | Packet #133
[SENSOR] Published -> Temp: 32.7 °C | Hum: 73.7% | Packet #134

[CMD] Inbound packet on topic: esp32/commands
[CMD] Output updated -> RED: ON | YELLOW: OFF | GREEN: ON (Manual Mode)
[SENSOR] Published -> Temp: 31.8 °C | Hum: 72.3% | Packet #135
[SENSOR] Published -> Temp: 32.5 °C | Hum: 72.3% | Packet #136
[SENSOR] Published -> Temp: 33.8 °C | Hum: 73.1% | Packet #137
[SENSOR] Published -> Temp: 34.3 °C | Hum: 72.1% | Packet #138
[SENSOR] Published -> Temp: 35.0 °C | Hum: 72.2% | Packet #139
```

Kết quả: Hệ thống có khả năng thu thập dữ liệu cảm biến và điều khiển thiết bị phần cứng theo thời gian thực, hoàn thiện luồng hoạt động của một hệ thống IoT đầy đủ.

Tuần 10: Hoàn thiện Hệ thống Cảnh báo & Tối ưu Hóa API

Mục tiêu: Tăng khả năng phản ứng trước các bất thường về môi trường và hoàn thiện chất lượng API.

Công việc đã thực hiện:

- Xây dựng cơ chế cảnh báo ngưỡng trong TelemetryMqttController:
 - tự động kiểm tra nhiệt độ/độ ẩm ngay khi nhận gói tin MQTT,
 - nếu chỉ số vượt ngưỡng an toàn, ví dụ nhiệt độ > 38°C hoặc độ ẩm < 30%, hệ thống lập tức kích hoạt sự kiện cảnh báo critical_alert qua WebSocket để người dùng kịp thời xử lý.
- Chuẩn hóa DTO đầu vào:
 - sửa lỗi đặt tên trường (garderID → gardenId) và ép kiểu số nguyên với class-validator (@IsInt, @Min(1)),
 - bổ sung ràng buộc @Min(1) cho các trường ID trong DTO nghiệp vụ.
- Chuẩn hóa trạng thái HTTP và tài liệu Swagger:
 - endpoint POST /device/command trả về 200 OK (thay vì 201 mặc định) và được khai báo rõ ràng trong Swagger,
 - bổ sung đầy đủ @ApiResponse cho từng endpoint để tài liệu API không còn mô tả thiếu (undocumented).
- Sửa lỗi cảnh báo route legacy: chuyển wildcard /api/* sang dạng named parameter *path trong cấu hình middleware, loại bỏ cảnh báo từ path-to-regexp.

Kết quả: Hệ thống phản ứng nhanh với các bất thường môi trường, API đồng bộ và tài liệu hóa đầy đủ.

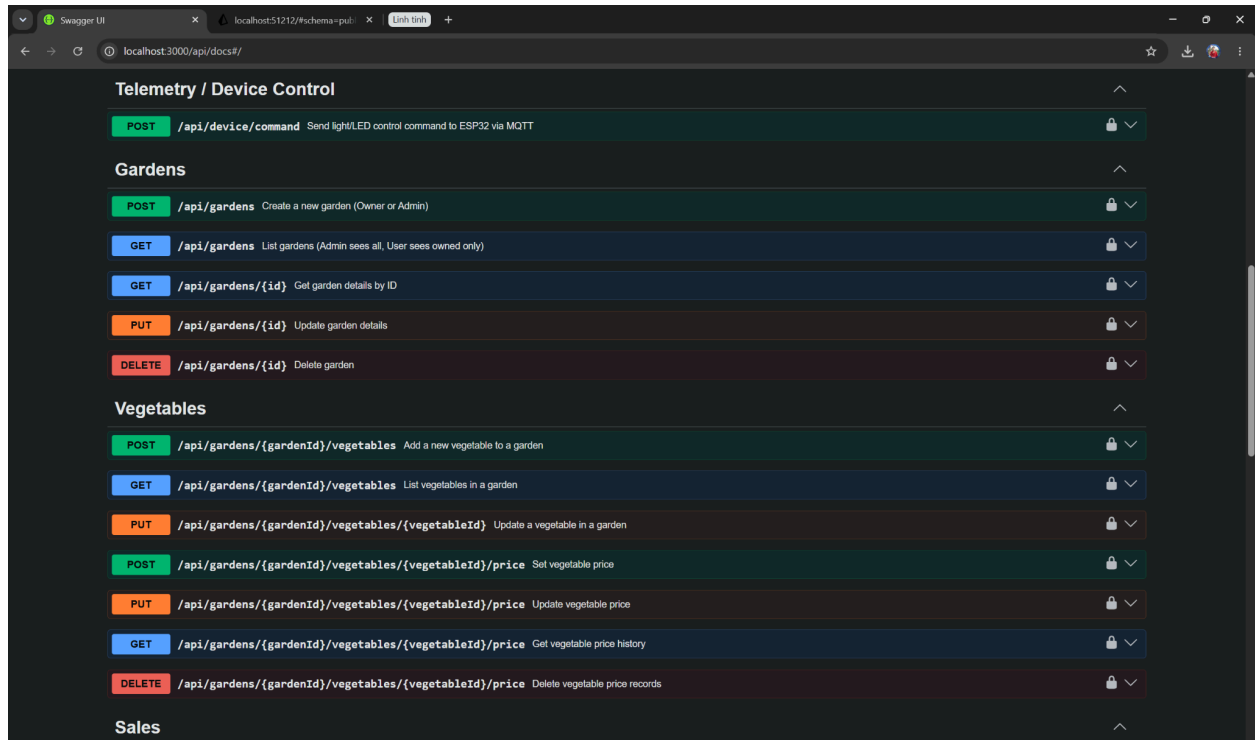
Tuần 11: Tài liệu hóa Chuẩn OpenAPI & Kiểm thử Tự động

Mục tiêu: Hoàn thiện tài liệu kỹ thuật và đảm bảo chất lượng hệ thống.

Công việc đã thực hiện:

- Chuẩn hóa giao diện Swagger UI tại /api/docs (với global prefix /api) gồm:
 - đầy đủ schema DTO (kèm example và mô tả),
 - mô tả chi tiết mã lỗi HTTP cho từng endpoint,
 - nút xác thực Bearer Token (JWT) để kiểm thử trực tiếp trên trình duyệt.
- Xây dựng bộ Unit Test với Jest:
 - mock PrismaService,
 - kiểm thử logic nghiệp vụ tại AuthService, GardenService, VegetableService, SaleService (bao gồm các trường hợp lỗi: khu vườn không tồn tại, không đủ quyền sở hữu, ADMIN bị chặn khi thêm nông sản/tạo giao dịch bán hàng vào khu vườn của người dùng khác, bán vượt kho, chưa set giá),
 - kiểm thử controller với service được mock (AuthController, GardenController, VegetableController, SaleController).
- Cập nhật bộ sưu tập postman_collection.json đồng bộ hoàn toàn với cấu trúc route mới:
 - các biến baseUrl, accessToken, gardenId, vegetableId,
 - chuỗi luồng kiểm thử: đăng nhập → tạo vườn → tạo rau → set giá → bán hàng → xem doanh thu → điều khiển thiết bị.

ẢNH 7: Giao diện Swagger UI tại <http://localhost:3000/api/docs>



ẢNH 8: Kết quả unit test Jest (npm run test và e2e test)

```
PS D:\GitHub\YootexIOT_Intern\week7\my-app> npm run test
Test Suites: 16 passed, 16 total
Tests:       75 passed, 75 total
Snapshots:   0 total
Time:        4.292 s
Ran all test suites.
```

```
PS D:\GitHub\YootexIOT_Intern\week7\my-app> npm run test:e2e
Test Suites: 2 passed, 2 total
Tests:       43 passed, 43 total
Snapshots:   0 total
Time:        7.521 s
Ran all test suites.
```

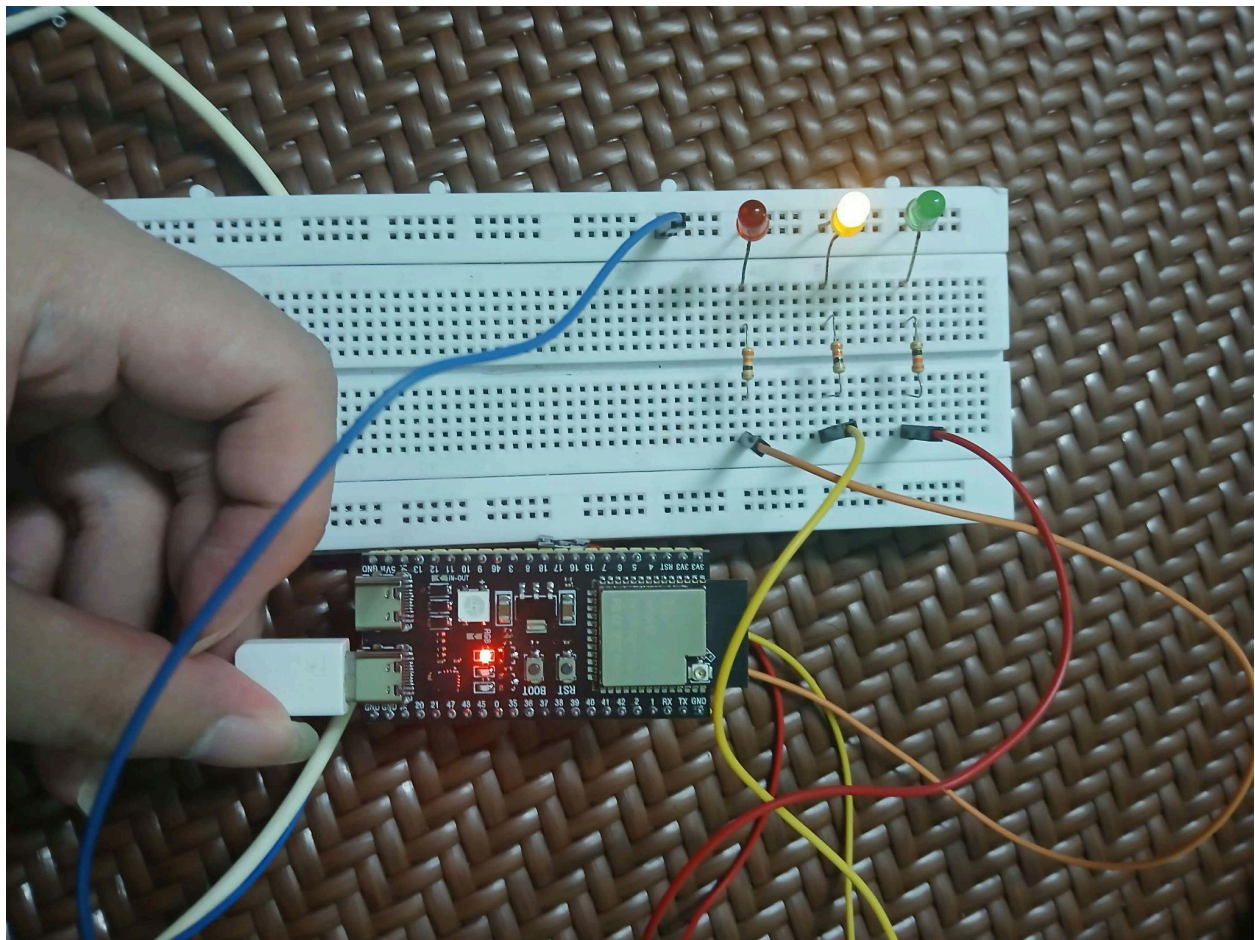
Kết quả: Hệ thống hoàn thiện từ tài liệu đến mã nguồn, có bộ kiểm thử tự động và công cụ kiểm thử API trực quan.

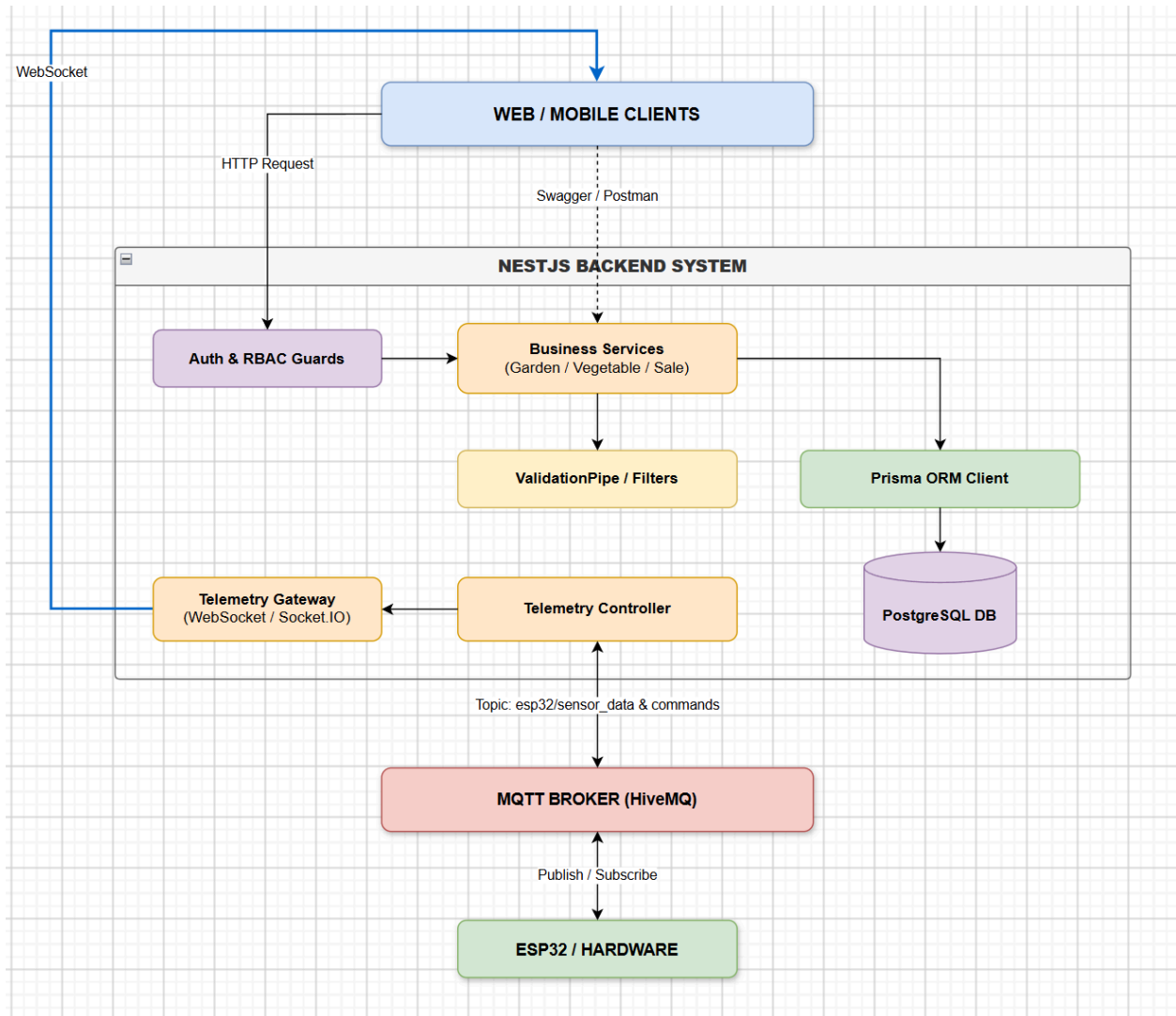
III. SƠ ĐỒ LƯỒNG DỮ LIỆU TOÀN HỆ THỐNG

Diễn giải luồng dữ liệu:

- Client giao tiếp với backend qua 3 kênh:
 - HTTP cho các nghiệp vụ CRUD,
 - WebSocket để nhận dữ liệu thời gian thực,
 - Swagger/Postman để kiểm thử API.
- Backend xác thực và phân quyền trước khi xử lý nghiệp vụ; dữ liệu được ghi vào PostgreSQL qua Prisma ORM.
- Thiết bị ESP32 gửi dữ liệu cảm biến lên MQTT Broker; backend lắng nghe topic esp32/sensor_data, kiểm tra ngưỡng an toàn, lưu trữ dữ liệu và phát sóng tới client qua WebSocket.
- Lệnh điều khiển từ client đi qua backend, được publish xuống topic esp32/commands để thiết bị thực thi (ví dụ: bật/tắt đèn).

ẢNH 9: Sơ đồ luồng dữ liệu toàn hệ thống





Chế độ	Đèn LED	Chân GPIO	Trạng thái kích hoạt	Ý nghĩa hệ thống
Tự động (Auto Mode)	Đỏ (Red)	GPIO 18	Nhiệt độ: Temp > 38.0°C hoặc Độ ẩm: Humi < 30.0%	Mức Nguy hiểm (Critical): Môi trường vượt ngưỡng an toàn, backend kích hoạt cảnh báo.
	Vàng (Yellow)	GPIO 17	Nhiệt độ: 32.0°C < Temp ≤ 38.0°C và Độ ẩm: Humi ≥ 30.0%	Mức Cảnh báo sớm (Warning): Nhiệt độ tăng cao cục bộ tại vườn nhưng chưa đến mức báo động đỏ.

	Xanh (Green)	GPIO 16	Nhiệt độ: Temp $\leq 32.0^{\circ}\text{C}$ và Độ ẩm: Humi $\geq 30.0\%$	Mức An toàn (Safe): Điều kiện môi trường lý tưởng cho cây trồng phát triển.
Thủ công (Manual Mode)	Cả 3 đèn	GPIO 18, 17, 16	Nhận lệnh điều khiển qua topic esp32/commands	Ghi đè thủ công: Bật/tắt độc lập theo giá trị payload (On/Off) từ Swagger UI hoặc Mobile/Web App, vô hiệu hóa tạm thời logic cảm biến tự động.
Khởi động (Self-test)	Cả 3 đèn	GPIO 18, 17, 16	Khi vừa cấp nguồn hoặc bấm nút Reset (RST)	Kiểm tra phản ứng: Chớp đồng thời -> sáng tuần tự Đỏ - Vàng - Xanh -> chớp nhanh.

IV. KẾT QUẢ KIỂM THỬ VÀ ĐÁNH GIÁ CHỨC NĂNG

1. Bảng Tổng hợp Kết quả Thực thi API Nghiệp vụ

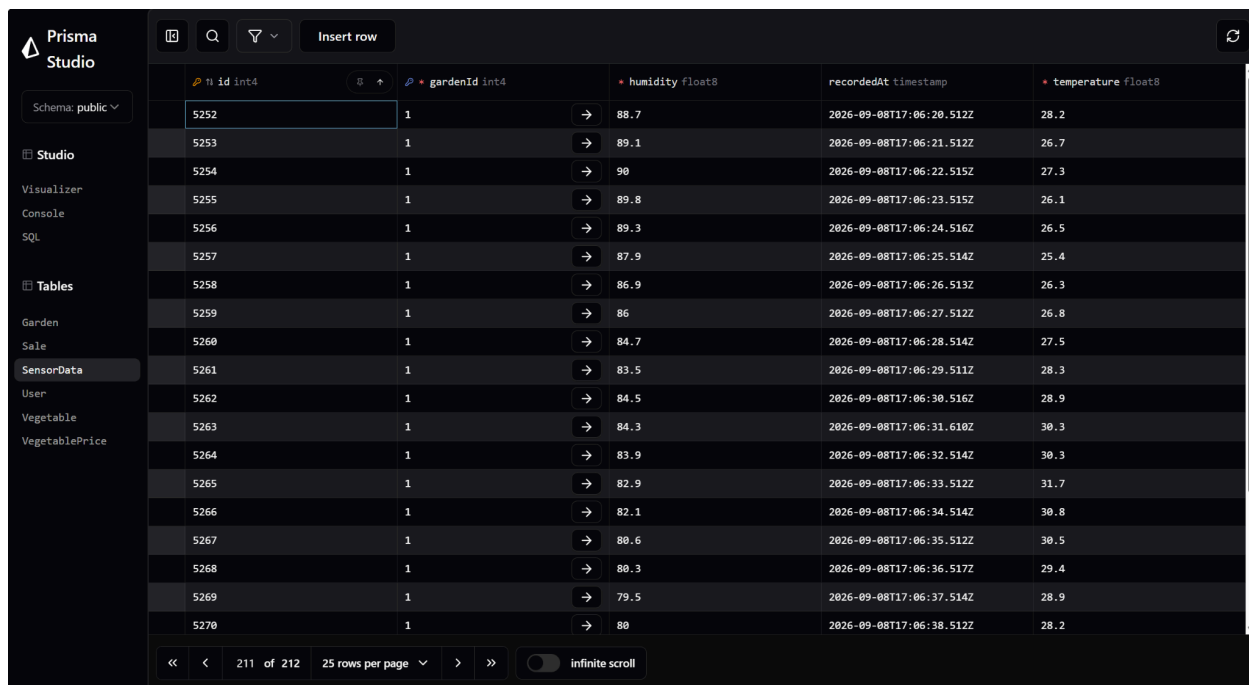
Nhóm chức năng	Endpoint	Phương thức	Dữ liệu đầu vào (DTO) / Tham số	Kết quả kiểm thử
Xác thực	/api/auth/login	POST	email, password	200 OK – Trả về JWT Token
Khu vườn	/api/gardens	POST	name	201 Created – Tạo vườn gắn với userId
Khu vườn	/api/gardens	GET	Bearer Token (User / Admin)	200 OK – Lọc chính xác theo quyền sở hữu
Khu vườn	/api/gardens/:id	GET	Bearer Token	200 OK – Admin xem được mọi vườn, User chỉ xem vườn của mình (403 nếu không thuộc quyền)
Nông sản	/api/gardens/:gardenId/vegetables	POST	name, importQty (gardenId lấy từ URL)	201 Created – Thêm rau vào vườn của chính mình
Nông sản (vườn khác – USER)	/api/gardens/:gardenId/vegetables	POST	name, importQty	403 Forbidden – Không phải chủ sở hữu vườn

Nông sản (vườn khác – ADMIN)	/api/gardens/:gardenId/vegetables	POST	name, importQty	403 Forbidden – Chỉ chủ vườn được thao tác nghiệp vụ, ADMIN bị từ chối
Nông sản	/api/gardens/:gardenId/vegetables	GET	gardenId	200 OK – Danh sách rau của đúng khu vườn
Giá rau	/api/gardens/:gardenId/vegetables/:vegetableId/price	POST	price: 25000	201 Created – Lưu bản ghi giá mới
Giá rau	/api/gardens/:gardenId/vegetables/:vegetableId/price	PUT	price: 28000	200 OK – Cập nhật giá mới nhất
Giá rau	/api/gardens/:gardenId/vegetables/:vegetableId/price	DELETE	vegetableId	200 OK – Xóa toàn bộ lịch sử giá
Bán hàng	/api/sales	POST	gardenId, vegetableId, quantity: 20	201 Created – Trừ tồn kho, lưu hóa đơn
Bán hàng (vượt kho)	/api/sales	POST	quantity: 9999	400 Bad Request – Báo lỗi thiếu hàng
Bán hàng (sai vườn)	/api/sales	POST	vegetableId không thuộc gardenId	400 Bad Request – Nông sản không thuộc khu vườn yêu cầu
Bán hàng (vườn khác – ADMIN)	/api/sales	POST	gardenId, vegetableId, quantity	403 Forbidden – Nghiệp vụ bán hàng chỉ chủ vườn thực hiện
Khu vườn (quản trị – ADMIN)	/api/gardens/:id	DELETE	Bearer Token (Admin)	200 OK – ADMIN xóa được vườn của mọi user phục vụ quản trị hệ thống
Doanh thu	/api/all/price?period=day	GET	period=day	200 OK – Tổng hợp doanh thu chuẩn xác
Điều khiển IoT	/api/device/comm and	POST	gardenId, userId, led1State	200 OK – Đẩy lệnh thành công xuống MQTT

3. Đánh giá về độ chính xác dữ liệu

- Các ràng buộc nghiệp vụ ($\text{soldQty} \leq \text{importQty}$) được kiểm tra chặt chẽ, mọi trường hợp bán vượt kho đều bị hệ thống từ chối với mã lỗi phù hợp.
- Doanh thu tính toán theo nhiều chu kỳ (day, week, month) cho kết quả nhất quán, đảm bảo độ tin cậy cho công tác thống kê.
- Dữ liệu cảm biến được lưu trữ đầy đủ, kèm mốc thời gian ghi nhận, phục vụ truy xuất lịch sử khi cần.

ẢNH 11: Dữ liệu kiểm chứng trong PostgreSQL (Prisma Studio)



id int4	gardenId int4	humidity float8	recordedAt timestamp	temperature float8
5252	1	88.7	2026-09-08T17:06:20.512Z	28.2
5253	1	89.1	2026-09-08T17:06:21.512Z	26.7
5254	1	90	2026-09-08T17:06:22.515Z	27.3
5255	1	89.8	2026-09-08T17:06:23.515Z	26.1
5256	1	89.3	2026-09-08T17:06:24.516Z	26.5
5257	1	87.9	2026-09-08T17:06:25.514Z	25.4
5258	1	86.9	2026-09-08T17:06:26.513Z	26.3
5259	1	86	2026-09-08T17:06:27.512Z	26.8
5260	1	84.7	2026-09-08T17:06:28.514Z	27.5
5261	1	83.5	2026-09-08T17:06:29.511Z	28.3
5262	1	84.5	2026-09-08T17:06:30.516Z	28.9
5263	1	84.3	2026-09-08T17:06:31.610Z	30.3
5264	1	83.9	2026-09-08T17:06:32.514Z	30.3
5265	1	82.9	2026-09-08T17:06:33.512Z	31.7
5266	1	82.1	2026-09-08T17:06:34.514Z	30.8
5267	1	80.6	2026-09-08T17:06:35.512Z	30.5
5268	1	80.3	2026-09-08T17:06:36.517Z	29.4
5269	1	79.5	2026-09-08T17:06:37.514Z	28.9
5270	1	80	2026-09-08T17:06:38.512Z	28.2

V. KẾT LUẬN

1. Kết luận

Sau giai đoạn thực tập từ Tuần 7 đến Tuần 11, em đã hoàn thành các mục tiêu chính của đề tài:

- xây dựng thành công hệ thống backend đa giao thức với đầy đủ chức năng quản lý nghiệp vụ,
- tích hợp thành công luồng dữ liệu IoT từ thiết bị phần cứng lên hệ thống và ngược lại,
- hoàn thiện hệ thống cảnh báo môi trường theo thời gian thực, tăng cường bảo mật và chuẩn bị đầy đủ tài liệu, kiểm thử tự động.

Quá trình thực hiện đề tài đã giúp em củng cố kiến thức về lập trình backend, cơ sở dữ liệu, IoT, xử lý thời gian thực và kiểm thử – những kỹ năng nền tảng quan trọng cho công việc trong lĩnh vực phát triển phần mềm sau này.

2. Hướng phát triển

Trong tương lai, hệ thống có thể được mở rộng thêm:

- tích hợp tầng Cache (Redis) cho các truy vấn doanh thu và dữ liệu cảm biến nhằm giảm tải đọc lên PostgreSQL,
- phân tích dữ liệu lớn và dự báo điều kiện môi trường,
- ứng dụng trí tuệ nhân tạo để hỗ trợ đưa ra khuyến nghị canh tác,
- đóng gói Docker (Dockerfile + docker-compose) để triển khai toàn bộ hệ sinh thái NestJS, PostgreSQL, MQTT Broker,
- mở rộng hỗ trợ nhiều loại thiết bị IoT và giao thức truyền thông khác nhau,
- phát triển ứng dụng di động để tăng tính tiện lợi cho người dùng.